EXERCICE PRATIQUE – GESTION DES EXCEPTIONS EN JAVA

Master 1 – Programmation Java | Système sécurisé de virements bancaires

Durée estimée : 45 à 60 minutes | Modalité : individuel ou en binôme

Objectifs pédagogiques : utiliser try/catch/finally, distinguer throw et throws, concevoir des exceptions personnalisées, gérer la propagation des exceptions et raisonner sur la cohérence des données après une défaillance.

Contexte

Vous développez le module de transactions d'une application bancaire. L'application effectue des virements entre comptes. Un virement peut échouer pour des raisons métier ou techniques. Votre objectif est de concevoir une solution robuste qui signale clairement les échecs et évite de laisser les données des comptes dans un état incohérent.

Classe de départ : BankAccount

```
public class BankAccount {
    private String accountNumber;
    private double balance;
    public BankAccount(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
    public String getAccountNumber() { return accountNumber; }
    public double getBalance() { return balance; }
    public void debit(double amount) { balance -= amount; }
    public void credit(double amount) { balance += amount; }
}
```

Partie 1 – Valider un virement

Créez une classe TransferService contenant une méthode transfer(...). Validez le virement avant de modifier l'un ou l'autre des comptes.

Règle	Comportement attendu
Montant ≤ 0	Rejeter le virement avec une exception appropriée.
Compte source ou destinataire null	Rejeter le virement.
Même compte source et destinataire	Rejeter le virement.
Montant > 10 000 €	Rejeter le virement : plafond dépassé.
Montant > solde du compte source	Rejeter le virement : fonds insuffisants.

Utilisez explicitement throw pour les conditions métier invalides. Déterminez quelles exceptions standard de Java conviennent à ce stade.

Partie 2 – Créer des exceptions métier

Créez la hiérarchie d'exceptions suivante. Vous pouvez ajouter des constructeurs et des attributs utiles si nécessaire.

```
Exception
  +-- BankingException
        +-- InsufficientFundsException
        +-- TransferLimitExceededException
        +-- InvalidAccountException
```

Modifiez transfer(...) pour déclarer les exceptions pertinentes avec throws. Utilisez throw dans la méthode lorsqu'une règle métier est violée.

Partie 3 – Gérer les erreurs côté appelant

Créez une classe Main utilisant Scanner. Demandez le montant d'un virement et exécutez-le. Utilisez plusieurs blocs catch afin d'afficher un message spécifique en cas de saisie invalide, de fonds insuffisants, de dépassement de plafond ou de compte invalide.

Ajoutez un bloc finally qui affiche : Traitement de la transaction terminé.

Question : pourquoi un bloc finally est-il approprié ici ? Quelles ressources faudrait-il normalement libérer dans une application réelle ?

Partie 4 – Analyse d'une défaillance et de la cohérence

Considérez l'implémentation suivante :

```
public void transfer(BankAccount source, BankAccount destination,
                     double amount) throws BankingException {
    source.debit(amount);
    // Imaginez qu'une panne inattendue survienne ici
    destination.credit(amount);
}
```

Analysez le code avant de le modifier. Supposez que le compte source possède 2 000 € et que le virement est de 500 €. Que se passe-t-il si une exception survient après debit() mais avant credit() ?

Répondez aux questions : (1) Quels sont les soldes finaux ? (2) try/catch a-t-il automatiquement annulé le débit ? (3) Le système est-il toujours cohérent ? (4) Comment repenser l'opération pour récupérer de manière sûre ?

Partie 5 – Mettre en œuvre une compensation

Améliorez le virement afin que, si l'opération de crédit échoue après un débit réussi, le compte source soit restauré. Votre implémentation doit conserver les informations relatives à l'erreur initiale au lieu de les ignorer silencieusement.

Ne vous contentez pas d'entourer toute la méthode avec catch (Exception e). Choisissez soigneusement le périmètre de gestion des exceptions et justifiez votre conception.

Partie 6 – Choix de conception

Déterminez comment le code appelant doit gérer les exceptions métier :

```
// Option A
catch (BankingException e) { ... }
// Option B
catch (InsufficientFundsException e) { ... }
catch (TransferLimitExceededException e) { ... }
catch (InvalidAccountException e) { ... }
```

Quelle option choisiriez-vous pour une application destinée aux utilisateurs ? Votre réponse changerait-elle pour une couche de journalisation ou un contrôleur API ? Justifiez.

Scénarios de test attendus

N°	Scénario	Résultat attendu
1	Virement de 500 € depuis un solde de 2 000 €	Succès ; soldes mis à jour.
2	Virement de 3 000 € depuis un solde de 2 000 €	InsufficientFundsException.
3	Virement de 12 000 €	TransferLimitExceededException.
4	Virement de −100 €	Virement rejeté.

N°	Scénario	Résultat attendu
5	Compte source identique au compte destinataire	InvalidAccountException.
6	L’utilisateur saisit « abc » comme montant	Erreur de saisie gérée ; pas de plantage.
7	Défaillance entre débit et crédit	État restauré ou protégé autrement.

Questions de discussion

1. Pourquoi une exception métier spécifique est-elle préférable à un simple throw Exception ?

2. Que se passe-t-il si BankingException hérite de RuntimeException plutôt que de Exception ?

3. Expliquez la différence entre lever, déclarer et gérer une exception.

4. Pourquoi l’utilisation systématique de catch(Exception e) peut-elle poser problème ?

5. Les exceptions techniques et les exceptions métier doivent-elles être gérées au même niveau ?

6. Le fait d’intercepter une exception garantit-il la cohérence des données ? Expliquez.

7. Dans un système bancaire de production, quel mécanisme remplacerait normalement la compensation manuelle pour les opérations en base de données ?

Livrables

À rendre : BankAccount.java, TransferService.java, toutes les classes d’exceptions personnalisées, Main.java et un court fichier README (5 à 10 lignes) expliquant votre stratégie de gestion des exceptions et votre solution de préservation de la cohérence.